Modules Are Programs

The global environment becomes an object

Gregory S. DeLozier, PhD

A Module File

Vertex program

```
 // file: modules/geometry.v
factor = 2;
function scale(x) { return factor * x };
function square(x) { return x * x };
sqrt = native("math.sqrt", ["x"]);
distance = native("math.hypot", ["a", "b"]);
```

Importing the Program

The module's **global environment becomes the object named `geometry`** in the importing program.
Vertex program

```
  geometry = import("modules/geometry.v");
print(geometry.factor);
print(geometry.square(5));
print(geometry.sqrt(9));
```

Prints `2`, `25`, then `3.0`.

The Module Object

Global binding	Object field
`factor = 2`	`geometry.factor`
`function scale(x) ...`	`geometry.scale`
`sqrt = native(...)`	`geometry.sqrt`

One Environment, Two Access Paths

Vertex program

```
  geometry = import("modules/geometry.v");
geometry.factor = 3;
print(geometry.scale(5));
```

Prints 15.

Tracing the Shared Binding

Reference	Destination
Importer’s `geometry`	Module globals object
`geometry.scale`	Vertex function value
`scale`’s retained environment	Same module globals object

A Stateful Module

Vertex program

```
 // file: modules/counter.v
count = 0;
function next() {
    set count = count + 1;
    return count;
};
function reset() { set count = 0 };
```

Independent Imports

Vertex program

```
  first = import("modules/counter.v");
second = import("modules/counter.v");
print(first.next());
print(first.next());
print(second.count);
```

Prints `1`, `2`, then `0`.

An Extracted Function

Vertex program

```
  counter = import("modules/counter.v");
next = counter.next;
print(next());
print(counter.count);
```

Prints 1, then 1.

Isolation

The module starts with fresh globals.
It can use supplied operations.
It cannot read arbitrary bindings from its caller.

Relative File Paths

Importing file	Requested path	Loaded file
`main.v`	`modules/outer.v`	`modules/outer.v`
`modules/outer.v`	`inner.v`	`modules/inner.v`

Program Completion

Module execution	Import result
Finishes normally	Global environment object
Empty program	Empty object
Top-level `return`	Program error
`exit`	Propagated evaluator status

Circular Imports

```
  a.v imports b.v
b.v imports a.v
```

The loader reports the cycle instead of recursing forever.

Globals and Host Bookkeeping

Module fields contain ordinary program bindings.
Source path and input/output bookkeeping stay in host attributes.

Copying Does Not Recreate Closures

Vertex program

```
  geometry = import("modules/geometry.v");
other = copy(geometry);
other.factor = 9;
print(other.scale(5));
print(other.factor);
```

Prints `10`, then `9`.

The Loader

1. Resolve the file path and check for a cycle.
2. Read and parse a program.
3. Evaluate in fresh module globals.
4. Return those globals on normal completion.

Create a Module: `modules/pricing.v`

Work in `chapter-13-native-functions-and-modules/code`. Create `modules/pricing.v`:

Vertex program

```
  rate = 0.25;
function total(amount) {
    return amount * (1 + rate);
};
```

These globals will become the imported object's fields.

Use the Module: `pricing-demo.v`

Create `pricing-demo.v` in the same `code` directory:

Vertex program

```
  pricing = import("modules/pricing.v");
print(pricing.rate);
print(pricing.total(100));
```

Run: `python3 runner.py pricing-demo.v`

Expected output: `0.25`, then `125.0`.

Change the Module Through Its Object

Append to `pricing-demo.v`, then run it again:

Vertex program

```
  pricing.rate = 0.50;
print(pricing.total(100));
quote = pricing.total;
print(quote(200));
```

Additional output: `150.0`, then `300.0`.

Import a Second Instance

Append to `pricing-demo.v`, then run it again:

Vertex program

```
  other = import("modules/pricing.v");
print(pricing.rate);
print(other.rate);
print(other.total(100));
```

Additional output: `0.5`, `0.25`, then `125.0`.

Considerations

Should imports cache instances?
Should helper globals stay public?
Which native values require adapters?